

Триггеры и курсоры

Триггеры

Триггер - это хранимая процедура особого типа, которая выполняет одну или несколько инструкций в ответ на событие.

По типу события триггеры делятся на два класса:

- DDL-триггеры
- DML-триггеры

DDL-триггеры

```
CREATE EVENT TRIGGER имя
ON событие
[ WHEN переменная_фильтра
  IN (значение_фильтра
    [, ... ] )
  [ AND ... ] ]
EXECUTE PROCEDURE
имя_функции ( )
```

```
CREATE OR REPLACE FUNCTION
abort_any_command()
RETURNS event_trigger
LANGUAGE plpgsql AS
$$
    BEGIN RAISE EXCEPTION 'команда %
                          отключена', tg_tag;
END;
$;$
```

```
CREATE EVENT TRIGGER abort_ddl
ON ddl_command_start
```

```
EXECUTE PROCEDURE abort_any_command();
```


События (event) DDL-триггера

- LOGIN
- DDL_COMMAND_START и DDL_COMMAND_END
- CREATE
- ALTER
- DROP
- COMMENT
- GRANT
- REINDEX
- REFRESH MATERIALIZED VIEW
- REVOKE
- SQL_DROP
- TABLE_REWRITE

```
CREATE EVENT TRIGGER init_session
ON login
EXECUTE FUNCTION init_session();
```

```
CREATE OR REPLACE FUNCTION init_session()
RETURNS event_trigger
LANGUAGE plpgsql AS
$$
DECLARE
    hour integer = EXTRACT('hour' FROM
                                current_time at
                                time zone 'utc');
BEGIN
    IF hour BETWEEN 2 AND 4 THEN
        RAISE EXCEPTION 'Login forbidden';
    END IF;
END;
$$
```

DML-триггеры

Для поддержания согласованности и точности данных используются декларативные и процедурные методы.

Триггеры применяются в следующих случаях:

- если использование методов декларативной целостности данных не отвечает функциональным потребностям приложения;
- если необходимо каскадное изменение через связанные таблицы в базе данных;
- если база данных денормализована и требуется способ автоматизированного обновления избыточных данных в нескольких таблицах;
- если необходимо сверить значение в одной таблице с неидентичным значением в другой таблице;
- если требуется вывод пользовательских сообщений и сложная обработка ошибок.

Классы DML-триггеров

Существуют два класса триггеров:

- **INSTEAD OF**. Триггеры этого класса выполняются в обход действий, вызывавших их срабатывание, заменяя эти действия. Например, обновление таблицы, в которой есть триггер **INSTEAD OF**, вызовет срабатывание этого триггера. В результате вместо оператора обновления выполняется код триггера.
- **AFTER/BEFORE**. Триггеры этого класса исполняются после или до

DML-триггеры

Синтаксис:

```
CREATE [ OR REPLACE ] CONSTRAINT ]
TRIGGER name { BEFORE | AFTER | INSTEAD OF } { event [ OR ... ] }
ON table_name [ FROM referenced_table_name ]
[ NOT DEFERRABLE |
[ DEFERRABLE ]
[ INITIALLY IMMEDIATE | INITIALLY DEFERRED ]
[ REFERRENCING { { OLD | NEW } TABLE [ AS ] transition_relation_name } [
... ] ] [
FOR [ EACH ] { ROW | STATEMENT } ]
[ WHEN ( condition ) ]
EXECUTE { FUNCTION | PROCEDURE } function_name ( arguments )
```

DML-триггеры

```
CREATE TRIGGER check_upd
BEFORE UPDATE
ON accounts
FOR EACH ROW
EXECUTE FUNCTION check_account_update();
```

```
CREATE OR REPLACE TRIGGER check_upd
BEFORE UPDATE OF balance
ON accounts
FOR EACH ROW
EXECUTE FUNCTION check_account_update();
```

```
CREATE TRIGGER log_update
AFTER UPDATE
ON accounts
FOR EACH ROW
WHEN (OLD.* IS DISTINCT FROM NEW.*)
EXECUTE FUNCTION log_account_update();
```

```
CREATE OR REPLACE FUNCTION
employee_insert_trigger_fnc()
RETURNS trigger AS
$$
    INSERT INTO "Employee_Audit" (
        "EmployeeId",
        "LastName",
        "FirstName",
        "UserName",
        "EmpAdditionTime")
    VALUES (NEW."EmployeeId",
        NEW."LastName",
        NEW."FirstName",
        current_user,
        current_date);
    RETURN NEW;
$$ LANGUAGE 'plpgsql';
```

```
CREATE TRIGGER employee_insert_trigger
AFTER INSERT ON "Employee"
FOR EACH ROW
EXECUTE PROCEDURE
employee_insert_trigger_fnc();
```


Alter для триггера

```
ALTER EVENT TRIGGER name DISABLE
ALTER EVENT TRIGGER name ENABLE [ REPLICA | ALWAYS ]
ALTER EVENT TRIGGER name OWNER TO
{ new_owner |
  CURRENT_ROLE |
  CURRENT_USER |
  SESSION_USER
}
ALTER EVENT TRIGGER name RENAME TO new_name
```

Курсоры

Операции в реляционной базе данных выполняются над множеством строк. Набор строк, возвращаемый инструкцией `SELECT`, содержит все строки, которые удовлетворяют условиям, указанным в предложении `WHERE`.

Курсоры можно классифицировать:

- По области видимости;
- По типу;
- По способу перемещения по курсору;
- По способу распараллеливания курсора

По области видимости

По области видимости имени курсора различают:

- **Локальные курсоры (LOCAL).** Область курсора локальна по отношению к пакету, хранимой процедуре или триггеру, в которых этот курсор был создан. Курсор неявно освобождается после завершения выполнения пакета, хранимой процедуры или триггера, за исключением случая, когда курсор был передан параметру OUTPUT.
- **Глобальные курсоры (GLOBAL).** Область курсора является глобальной по отношению к соединению.

По типу

По типу курсора различают:

- **Статические курсоры (STATIC).** Создается временная копия данных для использования курсором. Все запросы к курсору обращаются к указанной временной таблице, сам курсор не позволяет производить изменения.
- **Динамические курсоры (DYNAMIC).** Отображают все изменения данных, сделанные в строках результирующего набора при просмотре этого курсора. Значения данных, порядок, а также членство строк в каждой выборке могут меняться.
- **Курсоры, управляемые набором ключей (KEYSET).** Членство или порядок строк в курсоре не изменяются после его открытия. Набор ключей, однозначно определяющих строки, встроен в таблицу в базе данных.
- **Быстрые последовательные курсоры (FAST_FORWARD)**

По способу перемещения по курсору

- Последовательные курсоры (FORWARD_ONLY). Курсор может просматриваться только от первой строки к последней. Поддерживается только параметр выборки FETCH NEXT.
- Курсоры прокрутки (SCROLL). Перемещение осуществляется по группе записей как вперед, так и назад. Параметр SCROLL не может указываться вместе с параметром для FAST_FORWARD.^[L]_[SEP]

По способу распараллеливания курсора

По способу распараллеливания курсоров различают:

- **READ_ONLY**. Содержимое курсора можно только считывать.
- **SCROLL_LOCKS**. При редактировании данной записи вами никто другой вносить в нее изменения не может. Такую блокировку прокрутки иногда еще называют «пессимистической» блокировкой.
- **OPTIMISTIC**. Означает отсутствие каких бы то ни было блокировок. «Оптимистическая» блокировка предполагает, что даже во время выполнения вами редактирования данных, другие пользователи смогут к ним обращаться.

Объявление и открытие курсора

Синтаксис:

DECLARE

name [[NO] SCROLL]
CURSOR [(arguments)]
FOR query;

OPEN unbound_cursorvar [[NO] SCROLL]
FOR query;

DECLARE

cur1 refcursor;

cur2 **CURSOR FOR** SELECT *
FROM tenk1;

cur3 **CURSOR** (key integer) **FOR**
SELECT *
FROM tenk1
WHERE unique1 = key;

OPEN cur1 **FOR EXECUTE**

format('SELECT * FROM %I
WHERE col1 = \$1', tabname)

USING keyvalue;

Чтение данных из курсора

FETCH [direction { FROM | IN }] cursor
INTO target;

FETCH извлекает следующую строку (в указанном направлении) из курсора в целевую переменную.

Если подходящей строки нет, в целевую переменную записывается значение NULL(s). Как и в случае с SELECT INTO, можно проверить специальную переменную FOUND, чтобы узнать, была ли получена строка.

Если строка не получена, курсор перемещается в положение после последней строки или перед первой строкой, в зависимости от направления движения.

```
FETCH curs1 INTO rowvar;  
FETCH curs2 INTO foo, bar, baz;  
FETCH LAST FROM curs3 INTO x, y;  
FETCH RELATIVE -2 FROM curs4 INTO x;  
  
MOVE curs1;  
MOVE LAST FROM curs3;  
MOVE RELATIVE -2 FROM curs4;  
MOVE FORWARD 2 FROM curs4;  
  
UPDATE foo  
SET dataval = myval  
WHERE CURRENT OF curs1;  
  
CLOSE curs1;
```


Направление обработки данных курсора

Предложение `direction` может быть любым из вариантов, допустимых в команде `SQL FETCH`, за исключением тех, которые позволяют получить более одной строки.

- `NEXT`
- `PRIOR`
- `FIRST`
- `LAST`
- `ABSOLUTE count`
- `RELATIVE count`
- `FORWARD`
- `BACKWARD`.

Отсутствие `direction` эквивалентно указанию `NEXT`.

```
CREATE FUNCTION myfunc(refcursor, refcursor)
RETURNS SETOF refcursor AS $$
BEGIN
    OPEN $1 FOR SELECT * FROM table_1;
    RETURN NEXT $1;
    OPEN $2 FOR SELECT * FROM table_2;
    RETURN NEXT $2;
END;
$$ LANGUAGE plpgsql;

BEGIN;
    SELECT * FROM myfunc('a', 'b');
    FETCH ALL FROM a;
    FETCH ALL FROM b;
COMMIT;
```